

FORMATION PYTHON

J2

Fonctions et structures de données

Découper le code en fonctions réutilisables, et choisir la bonne structure : liste, dictionnaire, ensemble, tuple.

03-fonctions

04-structures-de-donnees

Objectifs de la journée



Des fonctions propres

def, return, arguments nommés, valeurs par défaut, docstrings.



Vérifier avec assert

Le calcul des pénalités, protégé par des tests minute.



Choisir sa structure

Liste, dict, set, tuple : chacune a son travail.



Le mini-annuaire

Comptages, dédoublement, NIF orphelins — en mémoire.

LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

Objets & pandas

J5

Intégration & projets

Fil rouge : pénalités de retard + annuaire.

Une fonction = une responsabilité



def nomme, return renvoie

Sans return, la fonction renvoie None — source classique de bug.



Annotations de types

def f(ca: int) -> str : documentation vivante, aide de VS Code.



Valeurs par défaut

plafond: float = 0.50 — l'appelant ne précise que l'essentiel.



La docstring dit le POURQUOI

Première ligne à l'impératif ; help(f) l'affiche.



tp_penalites.py

```
def taux_penalite(jours: int,
                  plafond: float = 0.50) -> float:
    """Taux applicable, plafonné."""
    if jours <= 0:
        return 0.0
    taux = 0.10 + 0.02 * mois(jours)
    return min(taux, plafond)

taux_penalite(45)          # → 0.12
taux_penalite(2000)       # → 0.50
```

assert : le contrat vérifié à chaque exécution



Un test en une ligne

assert f(30) == 0 : si faux, le programme s'arrête net.



Les bords d'abord

30 vs 31 jours, 0, le plafond : c'est là que ça casse.



Écrire le test AVANT

Le TP fournit les assert ; votre code doit les faire passer.



Demain, les vrais tests

assert est l'antichambre de pytest — même logique.



les bords du barème

```
# Règle : 10 % + 2 % par mois entamé  
# au-delà du 1er mois, plafonné à 50 %
```

```
assert mois_entames(30) == 0
```

```
assert mois_entames(31) == 1
```

```
assert mois_entames(61) == 2
```

```
assert taux_penalite(0) == 0.0
```

```
assert taux_penalite(2000) == 0.50
```

```
$ python tp_penalites.py # silence = OK
```

Listes et tuples : l'ordre compte



La liste : modifiable, ordonnée

append, sort, découpage lst[2:5] — l'outil à tout faire.



b = a ne copie PAS

Deux noms, une seule liste. Copier : b = a.copy().



Le tuple : figé

Coordonnées, paires (cle, valeur) — ce qui ne doit pas bouger.



Dépaquetage

code, nom = ("BIS", "Bissau") : deux affectations d'un coup.



listes & tuples

```
cas = [45, 620, 150]    # millions
cas.append(98)
cas.sort(reverse=True)
[620, 150, 98, 45]

# le piège de la copie
b = cas                # même liste !
b.append(999)
print(cas)             # 999 est là aussi

code, nom = ("BIS", "Bissau")
```

Dictionnaires : la fiche et le comptage



clé → valeur

La fiche contribuable naturelle : `c["nif"]`, `c["regime"]`.



`d[cle]` plante si absente

`d.get(cle, default)` jamais — LE réflexe des comptages.



Le motif du comptage

`compte[k] = compte.get(k, 0) + 1` — à connaître par cœur.



Liste de dictionnaires

Un fichier CSV en mémoire, c'est exactement ça.



tp_annuaire.py

```
c = {"nif": "100000001",  
     "region": "Bissau"}  
  
# comptage par région  
compte = {}  
for c in annuaire:  
    r = c["region"]  
    compte[r] = compte.get(r, 0) + 1  
  
{'Bissau': 2, 'Bafatá': 2, 'Gabú': 1}
```

Ensembles : dédoublonner, comparer



Unicité garantie

Un set ne garde qu'un exemplaire — et "in" y est instantané.



Le dédoublonnage type

Un set des NIF déjà vus + une liste des fiches gardées.



L'algèbre des ensembles

$a - b$: déclarants inconnus de l'annuaire. $a \& b$: communs.



Cas réel du dépôt

paiements.csv contient des NIF orphelins — on les trouvera ainsi.



dédoublonnage

```
vus, uniques = set(), []  
for c in annuaire:  
    if c["nif"] not in vus:  
        vus.add(c["nif"])  
        uniques.append(c)  
  
declarants = {"...001", "...009"}  
connus = {c["nif"] for c in uniques}  
  
orphelins = declarants - connus  
{'...009'}
```

Quelle structure pour quel besoin ?

Besoin	Structure	Exemple du fil rouge
Une suite ordonnée, modifiable	liste []	les déclarations d'un contribuable
Retrouver par clé	dict { : }	la fiche : nif → raison sociale
Unicité, appartenance, comparaison	set { }	NIF vus, NIF orphelins
Un enregistrement figé	tuple ()	(code_region, nom_region)
Compter par catégorie	dict + .get	contribuables par région
Fichier CSV en mémoire	liste de dicts	l'annuaire entier

Règle d'or : si vous écrivez une boucle pour chercher un élément, il vous fallait probablement un dict ou un set.

Pénalités et mini-annuaire

- 1 **tp_penalites.py** — faire passer tous les assert — les bords d'abord
- 2 **tp_annuaire.py** — comptage par région, dédoublonnage, orphelins, tri
- 3 **Relire son J1** — remplacer les répétitions par des fonctions
- 4 **Pour les rapides** — `declarations_manquantes(annee)` en avance de phase



Fichiers du jour

```
03-fonctions/  
    tp_penalites.py  
04-structures-de-donnees/  
    tp_annuaire.py
```



Un assert qui échoue vous DIT où chercher :
lisez la ligne, testez le bord à la main dans le
REPL.

À retenir

- ✓ **Fonctions** — petites, nommées, annotées — la pénalité se lit comme la règle métier.
- ✓ **assert** — le contrat minute : les bords (30/31 jours, plafond) sont verrouillés.
- ✓ **dict.get(k, 0) + 1** — le motif de comptage — vous le réécrirez cent fois.
- ✓ **set** — dédoublonner et comparer : les NIF orphelins tombent en une soustraction.
- ✓ **Structure = intention** — liste pour l'ordre, dict pour la clé, set pour l'unicité.



Demain — J3 : fichiers, formats et robustesse

Les VRAIS fichiers arrivent : CSV et JSON du dépôt, dates en quatre formats, exceptions — et votre premier paquet Python.